El Arca de Noé, Firmas MinHash y el Índice de Jaccard

Cristian Yecid Torres Rojas, Camilo Andres Arias Avilez,
Universidad Nacional de Colombia
Sede Bogotá
Bogotá D.C., Colombia
{crtorresr, cariasa}@unal.edu.co

***Resumen*—Este documento presenta una aplicación del ADT Tabla Hash en el campo de la biología genética. Mediante la segmentación de una secuencia de ADN por sus k-meros y, el uso de las firmas MinHash Sketch *(para comprimir los datos)* y el índice de Jaccard para definir la métrica de Mash, matemáticamente se puede construir una medida directa de mutación genética que permite discernir la similitud entre secuencias de *ADN*, con una complejidad computacional baja. Prescindiendo de los métodos de comparación de tipo *alignment - free*.**

***Index Terms*—ADT-Tablas Hash, MinHash Sketch, Indice de Jaccard, Comparación de Secuencias de ADN.**

I. Introducción y Planteamiento del Problema

Contexto General del Problema

En las últimas décadas, la biología computacional y la genómica han experimentado una transformación radical impulsada por las tecnologías de secuenciación de segunda y tercera generación (NGS y *Single-Molecule Long Reads*) . La acumulación masiva de datos genómicos en bases de datos públicas ha sobrepasado la capacidad de procesamiento de las herramientas computacionales tradicionales. En este escenario, la **genómica comparativa** se ha consolidado como una disciplina fundamental para la clasificación taxonómica, la filogenia, la identificación de patógenos y la detección de brotes epidémicos . El problema central en esta área consiste en medir el grado de similitud o la distancia mutacional entre dos o más genomas a partir de sus secuencias de nucleótidos.

Situación Específica a Resolver

Para determinar la similitud entre secuencias biológicas, el enfoque clásico se ha fundamentado en el **alineamiento de secuencias**. Sin embargo, el alineamiento exacto intenta resolver una correspondencia carácter por carácter a lo largo de millones o miles de millones de pares de bases. La situación específica que este proyecto aborda es la incapacidad de los métodos tradicionales basados en alineamiento completo para realizar comparaciones masivas e incondicionales de similitud genómica en tiempos computacionales viables. Se requiere, por tanto, un sistema capaz de estimar con precisión el grado de coincidencia entre genomas prescindiendo del costo de un alineamiento completo (*alignment-free methods*).

Relevancia del Problema

La estimación rápida de similitud genómica es crítica en entornos biológicos y médicos actuales. Permitir la búsqueda y agrupación de genomas en tiempo real habilita:

- La identificación oportuna de especies y cepas bacterianas o virales en muestras clínicas complejas .
- El agrupamiento de conjuntos metagenómicos masivos provenientes de muestras ambientales o microbiomas.
- El descarte y control de calidad de muestras secuenciadas erróneamente.

Desde la perspectiva de la ciencia de la computación, resolver este problema exige el diseño de estructuras de datos eficientes y algoritmos de reducción de dimensionalidad que compriman la información genómica sin perder su capacidad discriminativa.

Dificultades Computacionales Asociadas

El desarrollo de una solución computacional para la comparación genómica enfrenta tres barreras principales:

1. **Alta dimensionalidad y volumen de datos:** Un solo genoma puede contener desde millones (10^6) hasta miles de millones (10^9) de pares de bases. Representar estas secuencias de forma textual en memoria principal resulta impráctico.
2. **El cuello de botella cuadrático ($O(N^2)$):** Al intentar calcular la matriz de similitud par a par (*all-pairs comparison*) para un conjunto de N genomas, el número de parejas a comparar crece cuadráticamente como $\binom{N}{2} \approx \frac{N^2}{2}$. Si cada comparación individual requiere un alineamiento costoso, la tarea se vuelve intratable.
3. **Gestión de colisiones y rendimiento de dispersión:** La descomposición de secuencias en subcadenas de longitud fija o k-meros genera conjuntos masivos de claves. Indexar y contar la frecuencia o presencia de estos k-meros requiere diseñar Tablas Hash eficientes con funciones de dispersión adecuadas y un control riguroso sobre el factor de carga (α) para evitar que la complejidad durante las búsquedas crezca linealmente $\Theta(n)$.

Propuesta General del Equipo

Para abordar la problemática planteada, el equipo propone el desarrollo de un prototipo en **Python** fundamentado en la estimación de distancia genómica libre de alineamiento (*alignment-free*), basándose en los desarrollos teóricos y metodológicos del artículo *"Mash: fast genome and metagenome distance estimation using MinHash"* (Ondov et al., 2016).

El sistema reduce la alta dimensionalidad de las secuencias de ADN descomponiéndolas en subcadenas de longitud fija (k-meros) que son procesadas mediante una implementación propia del Tipo Abstracto de Datos (**ADT**) **Tabla Hash** y sus entradas asociadas, garantizando búsquedas e indexamiento en tiempo promedio $\Theta(1)$. A partir del método de firmas de tipo **MinHash sketch** se puede sintetizar la información genómica reduciendo drásticamente el consumo de memoria principal. Posteriormente, estos *sketches* se comparan para estimar de forma eficiente el **índice de Jaccard** y derivar finalmente la **distancia de Mash**, una medida directa de mutación que permite realizar comparaciones masivas de similitud genómica de forma rápida.

II. Objetivos

Objetivo General

Desarrollar un prototipo funcional en **Python** para la estimación rápida de distancia genómica basada en la métrica de Mash y el algoritmo MinHash, implementando y evaluando experimentalmente dos variantes del Tipo Abstracto de Datos (**ADT**) Tabla Hash (encadenamiento y direccionamiento abierto) sobre un conjunto de genomas de prueba, con el fin de cuantificar el impacto de la elección de la estructura de datos en el rendimiento y tiempo de ejecución del algoritmo.

Objetivos Específicos

1. **Implementar el ADT Tabla Hash en Python:** Diseñar e implementar desde cero dos esquemas de resolución de colisiones (encadenamiento mediante listas y direccionamiento abierto con probeo lineal) para el almacenamiento e indexamiento eficiente de los k-meros extraídos de las secuencias de ADN.
2. **Desarrollar los módulos de MinHash y distancia de Mash:** Construir las funciones para la generación de las firmas MinHash (*sketches*), la estimación del índice de Jaccard de conjuntos y la conversión matemática a distancia de Mash.
3. **Desarrollar el Módulo de Generación y Mutación Sintética:** Construir las funciones `generar_dna` y `mutar_dna` para sintetizar cadenas de ADN aleatorias sobre el alfabeto $\Sigma = \{A, C, G, T\}$ y simular variabilidad genética con una tasa de mutación puntual $r \in [0, 1]$.
4. **Realizar un análisis comparativo de rendimiento:** Medir y contrastar experimentalmente los tiempos de ejecución, comportamiento del factor de carga (α) y eficiencia de búsqueda entre el esquema de encadenamiento y el de direccionamiento abierto, demostrando la influencia de la estructura de datos elegida en la escalabilidad del sistema.

III. Alcance y Limitaciones

Alcance del Prototipo

El prototipo desarrollado en **Python** contempla la lectura y procesamiento de secuencias de ADN generadas mediante el algoritmo `generar_dna`. El sistema ejecuta la descomposición de secuencias en k-meros, su indexamiento mediante las dos variantes del ADT Tabla Hash (encadenamiento y direccionamiento abierto) y la generación de las firmas *MinHash sketches*. A partir de estos *sketches*, el programa calcula el índice de Jaccard y la distancia de Mash entre parejas de genomas, entregando como salida la matriz de distancias estimada, el comportamiento del factor de carga (α) y una comparativa de los tiempos de ejecución entre ambas estructuras de datos.

Limitaciones Técnicas y Computacionales

- **Gestión de Memoria RAM en Python:** El almacenamiento de la Tabla Hash y las estructuras auxiliares se realiza enteramente en memoria principal (RAM) sin apoyo de bases de datos persistentes en disco. Por lo tanto, la cantidad de k-meros únicos a indexar está limitada por la memoria RAM del equipo o entorno de ejecución (ej. Google Colab).
- **Ejecución Monohilo (Single-Thread):** La implementación en Python opera en un solo hilo de procesamiento en CPU sin hacer uso de multiprocesamiento distribuido ni aceleración por GPU, lo que condiciona la velocidad de ejecución al procesar volúmenes masivos de datos.
- **Tamaño de los Genomas de Prueba:** El alcance experimental se enfoca en genomas de tamaño pequeño a moderado (organismos virales y bacterianos en el rango de Kilobases a pocas Megabases), excluyendo genomas complejos (como el genoma humano de ~ 3.2 Gb) para evitar saturación de memoria durante la fase previa al *sketching*.

IV. Requerimientos Principales

Requerimientos Funcionales

1. **Generación Sintética y Simulación de Mutaciones:** El sistema debe incluir módulos para la generación estocástica de secuencias de ADN de longitud N (`generar_dna`) y la creación de secuencias derivadas aplicando una tasa de mutación puntual controlada $0 \leq r \leq 1$ (`mutar_dna`).
2. **Selección de Variantes del ADT Tabla Hash:** El sistema debe permitir instanciar y alternar entre dos estrategias de resolución de colisiones implementadas desde cero:

- `IndiceGenomicoChaining`: Encadenamiento dinámico mediante listas en cada casilla.
- `IndiceGenomicoOpenAddressing`: Direccionamiento abierto con sondeo lineal (*linear probing*).

3. **Extracción de los k-meros y Firmas MinHash Sketch:** En el programa se debe de desarrollar un módulo que permita construir las firmas MinHash Sketch de cada k-mero de una secuencia de *ADN* dada. Con el fin de que posteriormente se pueda hacer su respectivo análisis de similitud mediante la metrica de **Mash**.
4. **Reporte de Desempeño:** El programa debe imprimir en pantalla o consola la matriz de distancias resultantes, el factor de carga final (α) de la tabla hash y el tiempo total de ejecución.

Requerimientos No Funcionales

1. **Eficiencia de Búsqueda:** Las operaciones del ADT Tabla Hash deben garantizar un costo promedio de $\Theta(1)$ en acceso y búsqueda bajo dispersión uniforme.
2. **Modularidad y Programación Orientada a Objetos:** La solución debe construirse de forma modular en Python, separando las clases de la estructura de datos, las entradas de la tabla y la lógica del algoritmo MinHash.
3. **Portabilidad:** El código debe ser ejecutable en entornos estándar de Python 3 (como Google Colab o ejecuciones locales en terminal) sin requerir librerías externas complejas para la estructura de datos.

V. Trabajo Relacionado Inicial

El desarrollo del presente prototipo se fundamenta en tres referentes teóricos y metodológicos clave de la literatura en estructuras de datos, minería de datos masivos y biología computacional:

1. **Estimación de Distancia Genómica basada en MinHash (Ondov et al., 2016):** En el artículo *"Mash: fast genome and metagenome distance estimation using MinHash"*, Ondov et al. introducen una herramienta que adapta la técnica de MinHash para la reducción de dimensionalidad en genómica masiva. El método reduce secuencias biológicas extensas a pequeñas firmas dígitales (*sketches*) mediante la extracción de k-meros. A partir de estos *sketches*, *Mash* estima eficientemente el índice de Jaccard y deriva una distancia mutacional probabilística (*distancia de Mash*) que aproxima la Identidad Promedio de Nucleótidos (ANI) sin necesidad de realizar alineamientos globales costosos.

   Relación con el proyecto: Este trabajo constituye el modelo metodológico principal del proyecto, el prototipo desarrollado en Python implementará el algoritmo de *sketching*, el cálculo del índice de Jaccard y la conversión hacia la distancia de Mash para comparar los genomas de prueba.

2. **Fundamentos de Muestreo y Medida de Semejanza (Broder, 1997):** En su trabajo seminal *"On the resemblance and containment of documents"*, Andrei Broder establece el marco matemático para evaluar la similitud entre grandes colecciones de datos reduciendo el problema a operaciones de intersección de conjuntos de subcadenas (*shingles*). Broder demuestra matemáticamente que la probabilidad de que dos conjuntos compartan el mismo valor hash mínimo (*minhash*) equivale exactamente a su índice de Jaccard.

   Relación con el proyecto: Aporta el sustento probabilístico para el uso de perfiles comprimidos (*bottom-k sketches*). En lugar de trabajar con texto plano, la propuesta aplica el modelo teórico de Broder sobre los conjuntos de k-meros obtenidos del ADN.
3. **Minería de Datos Masivos y Reducción de Dimensionalidad (Leskovec et al., 2019):** En el texto *"Mining of Massive Datasets"*, los autores analizan los desafíos computacionales asociados a la búsqueda de elementos similares en grandes volúmenes de información, destacando la barrera del crecimiento cuadrático $O(N^2)$ en comparaciones par a par. El capítulo 3 aborda la tokenización de secuencias, la construcción de matrices de firmas y la preservación de la similitud en espacios de alta dimensionalidad.

   Relación con el proyecto: Justifica el diseño algorítmico del sistema, aportando los argumentos teóricos de por qué la combinación de k-meros, funciones de hash y firmas de MinHash permite evitar la comparación exhaustiva letra por letra entre genomas.

VI. Diseño de la Solución

Arquitectura General del Sistema

El prototipo se organiza en cuatro capas secuenciales en **Python**:

a) **Capa de Generación Sintética y Simulación de Mutaciones:** Mediante la función `generar_dna(longitud)`, el sistema sintetiza una secuencia de ADN original G_A de tamaño N. Posteriormente, mediante `mutar_dna(secuencia, tasa_mutacion)`, genera una variante genómica G_B alterando aleatoriamente nucleótidos con una probabilidad $r \in [0, 1]$.

b) **Capa de Construcción de los k-meros e Implementación del ADT Tabla Hash:** Genera el conjunto de los k-meros de secuencias de *ADN* dadas. Cada k-mero es procesado por la función hash e insertado en la estructura elegida (`IndiceGenomicoChaining` o `IndiceGenomicoOpenAddressing`), guardando la clave y su lista de posiciones.

c) **Capa de Compresión (MinHash Sketches):** Selecciona los valores hash mínimos (*bottom-s sketch*) de cada tabla para construir las firmas de los genomas, $S(G_A)$ y $S(G_B)$.

d) **Capa Analítica (Índice de Jaccard y Distancia de Mash):** Evalúa la coincidencia de las firmas entre $S(G_A)$ y $S(G_B)$, estimando el índice de Jaccard $\hat{J}(S(G_A), S(G_B))$ y calculando la distancia de Mash $D(S(G_A), S(G_B))$ para contrastarla contra la tasa de mutación real r.

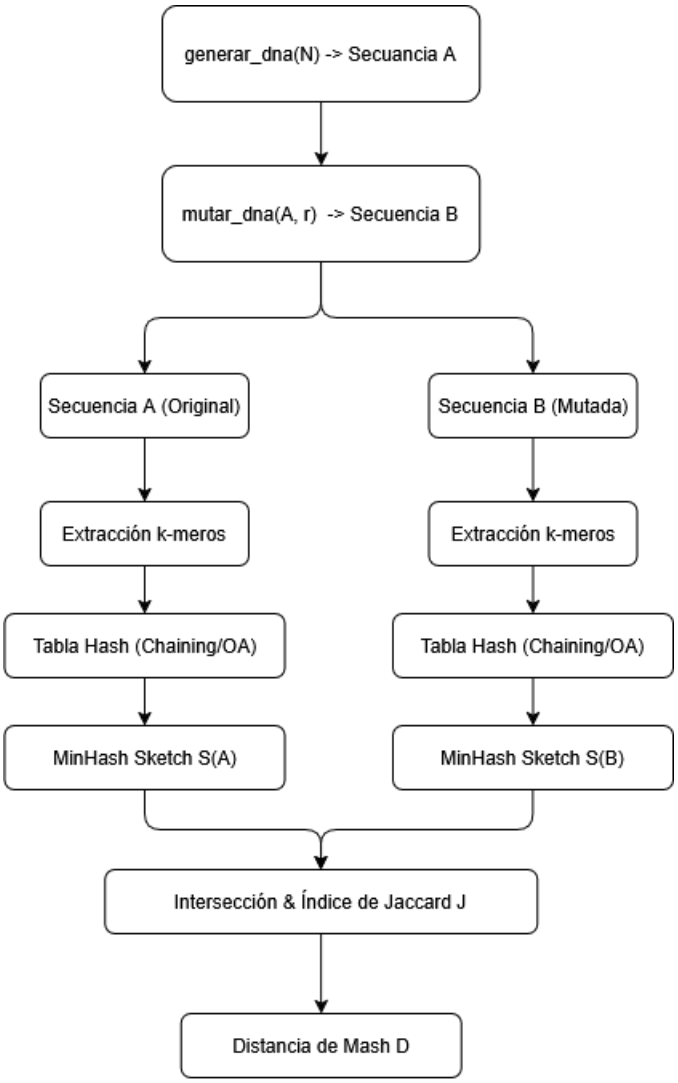


Figura 1. Diagrama de flujo

VII. Estructuras de Datos Seleccionadas

Para el desarrollo del prototipo se seleccionó como estructura principal el Tipo Abstracto de Datos (**ADT**) **Tabla Hash**, diseñado e implementado desde cero en **Python** mediante dos variantes de resolución de colisiones. Adicionalmente, se emplean estructuras auxiliares para la gestión de registros y se propone una estructura de soporte para la fase de reducción de dimensionalidad.

ADT Tabla Hash: Variante Encadenamiento (`IndiceGenomicoChaining`)

- **Justificación Teórica:** La técnica de hashing abierto o encadenamiento (*chaining*) separa el espacio de direccionamiento de la tabla del almacenamiento de los elementos. Cada casilla de la tabla funciona como el encabezado de una lista dinámica (*bucket*).
- **Implementación en el Código:** La estructura se representa mediante una lista principal de tamaño fijo m (`capacidad`), donde cada índice contiene una lista dinámica en la que se anexan las tuplas `(kmer, [posiciones])`.
- **Ventajas para el Problema Genómico:** Permite operar con factores de carga $\alpha = n/m \geq 1.0$ sin que la tabla colapse por falta de espacio. Es ideal cuando el número exacto de k-meros únicos n es desconocido antes del procesamiento.

ADT Tabla Hash: Variante Direccionamiento Abierto (`IndiceGenomicoOpenAddressing`)

- **Justificación Teórica:** En el hashing cerrado o direccionamiento abierto (*open addressing*), todos los elementos se almacenan directamente dentro del arreglo contiguo de la tabla. Ante una colisión, se aplica una secuencia de sondeo lineal (*linear probing*):

$$h(k, i) = (h'(k) + i) \pmod{m}$$

- **Implementación en el Código:** Se inicializa un arreglo plano de tamaño m lleno de valores nulos (`None`). Al insertar, el sistema recorre secuencialmente las casillas contiguas hasta encontrar una posición libre o el mismo k-mero para actualizar su lista de posiciones.
- **Ventajas y Limitaciones:** Ofrece una excelente localidad de referencia en la memoria caché de la CPU al trabajar sobre memoria contigua. Sin embargo, su desempeño se degrada rápidamente por agrupamiento primario conforme $\alpha \to 1.0$, exigiendo el control estricto $\alpha \leq 1.0$ e interrumpiendo la ejecución (`OverflowError`) si la capacidad se agota.

Estructuras Auxiliares Utilizadas

- **Arreglos Dinámicos (`list` en Python):** Utilizados para almacenar las secuencias de ADN generadas, la lista de casillas de la tabla y la lista de posiciones asociadas a cada k-mero. Garantizan acceso en $O(1)$ por índice y tiempo amortizado $O(1)$ al anexar elementos (`append`).
- **Tuplas Registros (`tuple`):** Utilizadas como nodos livianos e inmutables `(clave, valor)` que

vinculan cada k-mero con su respectiva lista de posiciones.

VIII. Estado de la implementación

A la fecha de la primera entrega, el proyecto presenta un **avance del** 45 % **en los aspectos teóricos**. Quedando pendiente la revisión del paper *On the resemblance and containment of documents - Adrei Z. Broder*, hacer la comparativa con los resultados obtenidos en el paper *Mash: fast genome and metagenome distance estimation using MinHash - Brian D. Ondov* y finalizar el estudio de la métrica de Mash presentada en el capítulo 3 del libro *Mining of Massive Datasets - Jure Leskovec.*

Por otro lado, en los **aspectos prácticos y computacionales**. Las estructuras de datos principales, Tablas Hash con direccionamiento abierto y de encadenamiento, ya se encuentran implementadas. Los módulos de generación de datos sintéticos ya se encuentran implementados.

Además, todas las implementaciones presentadas ya tienen su respectivo **análisis de complejidad computacional** y validación de su correcto funcionamiento *(mediante el uso de los módulos de generación de secuencias de ADN sintéticas).*

Componentes Desarrollados y Funcionales

a) **Módulo de Generación Sintética de ADN y Mutación Controlada (100 %):** Se implementaron exitosamente las funciones `generar_dna(longitud)` y `mutar_dna(secuencia, tasa_mutacion)`. Estas permiten sintetizar secuencias aleatorias sobre el alfabeto $\Sigma = \{A, C, G, T\}$ y generar variantes con tasas de mutación puntual controladas $0 \leq r \leq 1$.

b) **ADT Tabla Hash por Encadenamiento - `IndiceGenomicoChaining` (100 %):** Se construyó la clase en Python con soporte para resolución de colisiones mediante listas dinámicas en cada casilla. La estructura permite la inserción de k-meros, el registro acumulativo de sus posiciones de aparición dentro del genoma y la búsqueda en tiempo promedio $\Theta(1)$.

c) **ADT Tabla Hash por Direccionamiento Abierto - `IndiceGenomicoOpenAddressing` (100 %):** Se completó la implementación del hashing cerrado utilizando la secuencia de sondeo lineal (*linear probing*). La estructura gestiona la inserción directa de tuplas en un arreglo contiguo de memoria y cuenta con control de saturación mediante la excepción `OverflowError` cuando el factor de carga alcanza $\alpha = 1.0$.

d) **Módulo de Generación de los k-meros (100 %):** Se integró el algoritmo de generación de los k-meros de una secuencia de *ADN*, la cual obtiene todas las subcadenas de longitud k de la secuencia, para que posteriormente se puedan indexar dinámicamente en cualquiera de las dos variantes del ADT Tabla Hash.

IX. ¿Qué es una Hash Table y cuál es su propósito?

Una **Tabla Hash** (*Hash Table*) es una estructura de datos altamente eficiente diseñada para representar un conjunto dinámico que soporta las operaciones fundamentales de un diccionario: inserción (`INSERT`), búsqueda (`SEARCH`) y eliminación (`DELETE`).
A diferencia de las estructuras lineales donde la búsqueda toma tiempo $\Theta(n)$ o las estructuras basadas en árboles que requieren $O(\log n)$, una tabla hash permite realizar las operaciones del diccionario en un tiempo promedio de $\Theta(1)$ bajo suposiciones razonables.
Direccionamiento directo (*direct addressing*):

- **Tablas de Direccionamiento Directo:** Si el universo de claves posibles U es pequeño, se puede asignar una posición o *slot* en un arreglo $T[0 \dots m-1]$ para cada clave $k \in U$. El acceso a la clave k se hace directamente en $T[k]$ en tiempo $\Theta(1)$ en el peor caso.
- **El problema de la escala:** Cuando el universo de claves U es extremadamente grandes, asignar un arreglo de tamaño $|U|$ es impráctico o imposible. Además, si el conjunto de claves realmente almacenadas K es mucho menor que U, la gran mayoría del espacio asignado se desperdiciaría.
- **El mecanismo Hash:** En lugar de usar la clave k directamente como un índice del arreglo, la posición en la tabla se calcula mediante una **función hash** h:

$$h : U \to \{0, 1, \dots, m-1\}$$

La función h mapea la clave k al índice $h(k)$ dentro de una tabla T de tamaño m, reduciendo el requerimiento de memoria de $|U|$ a $\Theta(|K|)$ manteniendo el tiempo promedio de búsqueda en $O(1)$.

Conceptos Importantes

a) Colisiones (Collisions)

Dado que el universo de claves U es mayor que el tamaño de la tabla m ($|U| > m$), es matemáticamente inevitable que dos o más claves distintas mapeen a la misma ranura. Este fenómeno se denomina **colisión** ($h(k_1) = h(k_2)$ con $k_1 \neq k_2$).

b) Métodos de Resolución de Colisiones

a) **Encadenamiento (*Chaining*):** Coloca todos los elementos que colisionan en la misma posición dentro de una **lista enlazada**.
- La inserción `CHAINED-HASH-INSERT` se realiza al frente de la lista en tiempo con $\Theta(1)$ en el peor caso.
- La eliminación `CHAINED-HASH-DELETE` toma tiempo $\Theta(1)$ en el peor caso si las listas son **doblemente enlazadas**.

b) **Direccionamiento Abierto (*Open Addressing*):** Todos los elementos ocupan la misma tabla (sin listas externas). Si ocurre una colisión, el algoritmo busca secuencialmente mediante una **secuencia de sondeo** hasta encontrar una casilla vacía o con la marca especial `DELETED`. Técnicas principales de sondeo:
- **Prueba Lineal (*Linear Probing*):** $h(k,i) = (h'(k) + i) \bmod m$. Sufre de *agrupamiento primario* (*primary clustering*).
- **Prueba Cuadrática(*Quadratic Probing*):** $h(k,i) = (h'(k) + c_1 i + c_2 i^2) \bmod m$. Sufre de *agrupamiento secundario*.
- **Doble Hashing (*Double Hashing*):** $h(k,i) = (h_1(k) + i \cdot h_2(k)) \bmod m$. Produce el mejor rendimiento aproximándose al sondeo uniforme.

c) Factor de Carga (α)

Se define el **factor de carga** de una tabla de tamaño m que almacena n elementos como:

$$\alpha = \frac{n}{m}$$

Representa el número promedio de elementos almacenados por ranura. En encadenamiento, α puede ser menor, igual o mayor a 1; en direccionamiento abierto, nunca puede exceder 1 ($\alpha \leq 1$).

d) Funciones de Hash

- **Método de la División:** $h(k) = k \bmod m$. Se recomienda que m sea un número primo que no esté cerca de potencias de 2.
- **Método de la Multiplicación:** $h(k) = \lfloor m(kA \bmod 1) \rfloor$, donde A es una constante fraccionaria (Knuth sugiere $A \approx \frac{\sqrt{5}-1}{2} \approx 0.618033$).
- **Hashing Universal (*Universal Hashing*):** Selecciona la función de hash de forma **aleatoria** dentro de una clase diseñada de funciones H, garantizando matemáticamente un buen comportamiento promedio sin importar las claves que elija un adversario.

A continuación una implementación en Python en la cual se realiza un ejemplo de aplicación de las Hash tables

Descripción del Código y Análisis de Complejidad

Descripción General del Sistema

El código implementa dos variantes del Tipo Abstracto de Datos (ADT) Tabla Hash en Python aplicadas a la bioinformática: `IndiceGenomicoChaining` e `IndiceGenomicoOpenAddressing`. Ambas clases permiten registrar e indexar subcadenas de longitud fija (k-meros) junto con las posiciones en las que aparecen dentro de una secuencia genómica. La clase `IndiceGenomicoChaining` utiliza encadenamiento (*chaining*) o hashing abierto, manejando colisiones mediante listas dinámicas internas en cada casilla. Por su parte, la clase `IndiceGenomicoOpenAddressing` utiliza direccionamiento abierto (*open addressing*) o hashing cerrado con la técnica de sondeo lineal (*linear probing*), almacenando los elementos directamente en un arreglo contiguo de memoria y buscando secuencialmente la siguiente casilla disponible ante una colisión.

Análisis de Complejidad: Encadenamiento (`IndiceGenomicoChaining`)

En cuanto a la complejidad espacial, esta estructura requiere $\Theta(m + n)$, donde m es la capacidad de la tabla (`capacidad`) y n es el número de k-meros únicos registrados. Esto se debe a que exige reservar espacio para el arreglo principal más los elementos almacenados en las listas dinámicas de cada casilla. En términos de complejidad temporal, el cálculo del índice hash en la función `_hash` toma tiempo $\Theta(k)$, donde k es la longitud del k-mero (equivalente a $O(1)$ cuando k es constante). Para los métodos de inserción (`registrar_kmer`) y consulta (`buscar_posiciones`), el caso promedio bajo Hashing Uniforme Simple toma tiempo $\Theta(k + \alpha)$, siendo $\alpha = n/m$ el factor de carga de la tabla. En el peor caso —cuando todas las claves colisionan en la misma posición de la tabla— la complejidad temporal se degrada a $\Theta(k \cdot n)$ debido a que debe recorrer secuencialmente la lista de colisiones de tamaño n.

Análisis de Complejidad: Direccionamiento Abierto (`IndiceGenomicoOpenAddressing`)

La complejidad espacial para esta implementación es estrictamente $\Theta(m)$, dado que reserva un bloque de memoria contiguo y fijo de tamaño m inicializado con valores nulos, exigiendo que el factor de carga se mantenga en $\alpha \leq 1.0$ para evitar el desbordamiento de la tabla (*OverflowError*). Respecto a la complejidad temporal, la función `_hash` opera en $\Theta(k)$. Las operaciones de inserción (`registrar_kmer`) y búsqueda (`buscar_posiciones`) requieren en el caso promedio un tiempo de $\Theta\left(k \cdot \frac{1}{1-\alpha}\right)$, el cual se incrementa

conforme el factor de carga α se aproxima a 1. En el peor escenario —causado por el fenómeno de agrupamiento primario o cuando la tabla está completamente llena— el tiempo de ejecución se degenera a $\Theta(k \cdot m)$ al verse obligado a realizar un sondeo lineal a través de todas las casillas de la tabla.

X. La Paradoja del Cumpleaños y Funciones Hash

Un problema muy frecuente al analizar la *complejidad computacional de las funciones hash son las colisiones* que esta puede tener, es decir, que para dos valores diferentes del dominio su imágenes mediante la función hash sean iguales.

El análisis de las colisiones de funciones hash es uno de los pilares fundamentales al analizar nuestras implementaciones del ADT - Tabla hash. Así pues, para familiarizarnos con el tipo de argumentos probabilísticos presentados en los textos guía, vamos a presentar un problema equivalente a analizar la colisión de una función hash. Este problema se le conoce como **la paradoja del cumpleaños**

Paradoja del Cumpleaños: De un conjunto de $2 \leq n \leq 365$ personas. ¿Cuál es la probabilidad de que dos personas cumplan años el mismo día?

Sea $p(n)$ la probabilidad de que al menos dos personas cumplan años el mismo día en un grupo de $2 \leq n \leq 365$ personas. De esta forma por la ley del complemento sabemos que la probabilidad de que ninguna persona cumpla años el mismo día en un grupo el grupo de n personas es $1 - p(n)$.

En un conjunto de n personas, la primera persona tiene 365 días para tener su cumpleaños, para que la segunda persona no cumpla años el mismo día que la primera persona se necesita que su cumpleaños sea en los 364 días restantes, para que la tercer persona no cumpla años el mismo día que alguna de las personas anteriores se necesita que su cumpleaños sea en los 363 días restantes. Si seguimos así sucesivamente, para que la n-ésima persona no cumpla años el mismo día que alguna de las $n-1$ personas anteriores se necesita que su cumpleaños sea en los $365-(n-1)$ días restantes. Así pues, tenemos las siguiente fórmula para $1 - p(n)$:

$$\begin{aligned} 1 - p(n) &= \left(\frac{365}{365}\right)\left(\frac{364}{365}\right)\left(\frac{363}{365}\right)\cdots\left(\frac{365-(n-1)}{365}\right) \\ &= 1\left(1-\frac{1}{365}\right)\left(1-\frac{2}{365}\right)\cdots\left(1-\frac{n-1}{365}\right) \end{aligned}$$

De esta forma, la probabilidad de que al menos dos personas cumplan años el mismo día en un conjunto de $2 \leq n \leq 365$ personas es:

$$\boxed{p(n) = 1 - 1\left(1-\frac{1}{365}\right)\left(1-\frac{2}{365}\right)\cdots\left(1-\frac{n-1}{365}\right)}$$

Ahora, vamos a hacer una implementación en Python para ver el comportamiento de la probabilidad respecto al número de personas en el grupo. Donde obtenemos la siguiente gráfica:

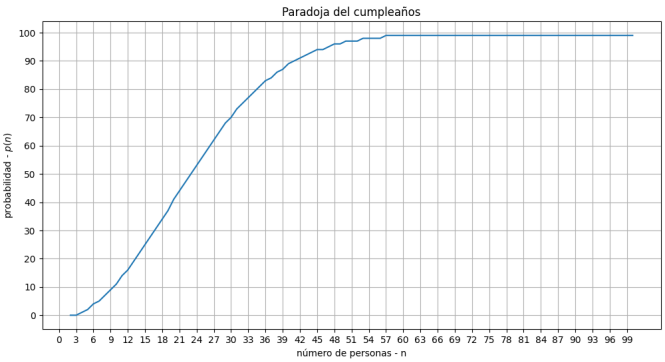


Figura 2. Paradoja del Cumpleaños

En la gráfica anterior se puede ver algo un poco particular:

$$p(23) \approx 50.7\,\% \quad , \quad p(41) \approx 90.3\,\%$$

Es decir, que en un grupo de al menos 23 personas vamos a tener un 50 % de probabilidad de que al menos dos personas cumplan años el mismo día y, ya en un grupo de 43 personas vamos a tener una probabilidad del 90 % *(es casi seguro que dos personas cumplan años el mismo día).*

A este comportamiento se le conoce como la paradoja del cumpleaños. Y este tipo de argumentos son los que vamos a encontrar al analizar las colisiones de las funciones hash que se implemente en el proyecto.

XI. Encontrando Items Similares con Firmas Hash

Como ya hemos discutido anteriormente, el problema fundamental con el que nos enfrentamos en el proyecto es el de comparar dos conjuntos de genomas con el fin de determinar su grado de similitud *(en un tiempo computacional razonable).*

Para abordar este problema, vamos a trabajar la teoría presentada en el libro **Mining of Massive Datasets - Jure Leskovec,et al.**, específicamente **el capítulo 3 - Finding Similar Items**. El recurso se puede encontrar en libre acceso en el siguiente

enlace Mining of Massive Datasets.

En primer lugar, vamos a definir que es matemáticamente el significado de que dos conjuntos A y B sean tengan un **grado de similitud alto**.

Definición: Dado un conjunto no vacío X y $A, B \subseteq X$ dos subconjuntos finitos, definimos el **índice de similitud de Jaccard** como:

$$\text{SIM}(A, B) := \frac{|A \cap B|}{|A \cup B|}$$

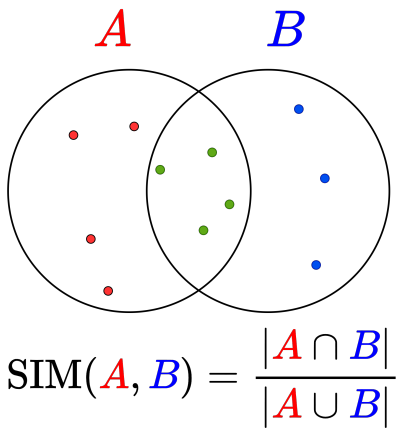


Figura 3. Índice de Similitud de Jaccard

Para ver el buen comportamiento del índice de Jaccard vamos a ver los siguientes teoremas:

Teorema: Sean $A, B \subseteq X$ dos subconjuntos finitos no vacíos, entonces tenemos la siguiente equivalencia:

$$\text{SIM}(A, B) = 1 \text{ si y solo si } A = B.$$

Demostración:
($\Longrightarrow$) Supongamos que $\text{SIM}(A, B) = 1$, entonces:

$$|A \cap B| = |A \cup B|$$

Tenemos la siguiente contenencia de conjuntos:

$$A \cap B \subseteq A \subseteq A \cup B \quad , \quad A \cap B \subseteq B \subseteq A \cup B$$

De esta forma, podemos concluir que:

$$|A \cap B| \leq |A| \leq |A \cup B| \quad , \quad |A \cap B| \leq |B| \leq |A \cup B|$$

Dado que por hipótesis $|A \cap B| = |A \cup B|$ entonces:

$$|A| = |A \cap B| = |A \cup B| = |B|$$

Y por la relación de contenencia presentada anteriormente, tenemos que:

$$A = B$$

($\Longleftarrow$) Si $A = B$ entonces su índice de similitud de Jaccard es:

$$\begin{aligned} \text{SIM}(A, B) &= \frac{|A \cap B|}{|A \cup B|} \\ &= \frac{|A|}{|A|} \\ &= 1 \end{aligned}$$

Teorema: Sean $A, B \subseteq X$ dos subconjuntos finitos no vacíos, entonces tenemos la siguiente equivalencia:

$$\text{SIM}(A, B) = 0 \text{ si y solo si } A \cap B = \emptyset.$$

Demostración:
($\Longrightarrow$) Supongamos que $\text{SIM}(A, B) = 0$, entonces por definición como A y B son conjuntos no vacíos:

$$|A \cap B| = 0$$

Esto ocurre si $A \cap B = \emptyset$.
($\Longleftarrow$) Supongamos que $A \cap B = \emptyset$ entonces su índice de similitud de Jaccard es:

$$\begin{aligned} \text{SIM}(A, B) &= \frac{|A \cap B|}{|A \cup B|} \\ &= \frac{0}{|A \cup B|} \\ &= 0 \end{aligned}$$

Estos dos últimos teoremas nos dicen que el comportamiento del índice de Jaccard está describiendo exactamente el grado de similitud de los dos conjuntos, *en el sentido de ver sus elementos comunes*.

Una propiedad importante que se utiliza comúnmente al analizar las propiedades del índice de Jaccard es **el principio de inclusión exclusión**. El cual presentamos en la siguiente figura.

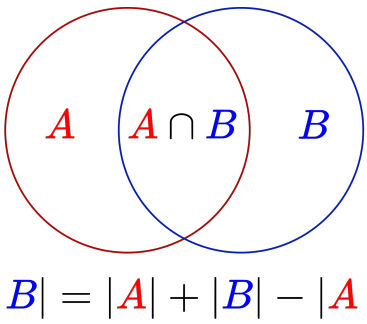


Figura 4. Principio de Inclusión - Exclusión

Ahora, nuestro objeto de estudio son los genomas, cadenas de bases nitrogenadas de tipo **A** (adenina), **G** (guanina),

C (citosina) y **T** (timina). Los cuales podemos modelar como una palabra finita sobre el alfabeto $\Sigma = \{A, G, C, T\}$.

Así pues, si g_1 es un genoma de alguna especie entonces en nuestro modelo tenemos que:

$$g_1 \in \Sigma^*$$

Donde Σ^* es la clausura de Kleene del alfabeto $\Sigma = \{A, G, C, T\}$.

A lo largo del proyecto, vamos a necesitar generar secuencias de *ADN* aleatoriamente con el fin de corroborar el correcto funcionamiento de nuestros algoritmos. Por lo tanto, en el siguiente enlace se puede ver la implementación Generador Aleatorio de ADN.

```
1 import random as rand
2
3 def generar_dna(longitud):
4     sec_dna = list()
5     bases = ['A','C','G','T']
6     for i in range(longitud):
7         sec_dna.append(rand.choice(bases))
8     return ''.join(sec_dna)
```

En la implementación se utiliza una estructura de dato de tipo *array* cuyos elementos son de tipo *char*. Esto con el fin de que al momento de hacer la operación sec_dna + = rand.choice(bases) se comporte como un Push Back en la estructura.

Para hacer el análisis de complejidad de nuestro algoritmo de generación aleatoria de secuencias de *ADN*, en una secuencia larga de operaciones de tipo Push Back, vemos que el tipo de política de *resize* que se emplea en Python es newCapacity $= \lceil \frac{9}{8} \cdot \text{capacity} \rceil$. Así pues, en virtud del análisis de costo amortizado realizado en clase para este tipo de políticas, sabemos que nuestra implementación tiene un costo amortizado de $O(1)$.

Por otro lado, también vamos a necesitar un algoritmo que nos permita mutar una secuencia de *ADN* dado un porcentaje de mutación.

Para esta implementación vamos a utilizar la librería *random* de Python la cual implementa un método que simula una variable aleatoria X que toma valores en el intervalo cerrado $[0, 1]$ bajo una distribución pseudo-uniforme con función de densidad:

$$p(x) = 1 \quad , \quad \text{para todo } 0 \leq x \leq 1$$

Así pues la probabilidad de que el valor de la variable aleatoria X tome un valor en el intervalo $[0, a]$ para algún $0 \leq a \leq 1$ es:

$$\begin{aligned} P(X \leq a) &= \int_0^a p(x)dx \\ &= \int_0^a 1dx \\ &= x\,|_0^a \\ &= a \end{aligned}$$

Y el valor esperado de la variable aleatoria X es:

$$\begin{aligned} \mathbb{E}(X) &= \int_0^1 x \cdot p(x)\ dx \\ &= \int_0^1 x\ dx \\ &= \frac{x^2}{2}\,|_0^1\ dx \\ &= \frac{1}{2} \end{aligned}$$

De esta forma, ya sabemos el comportamiento de la variable aleatoria X en una distribución uniforme en el intervalo cerrado $[0, 1]$. Y podemos hacer la siguiente implementación Secuencias de ADN Mutadas.

Para el análisis de complejidad de nuestra implementación vamos a suponer que las siguientes operaciones definidas en las lineas 8 y 7 se realizan en tiempo constante, es decir, tienen una complejidad computacional de $\theta(1)$:

```
1 import random as rand
2
3 def mutar_dna(secuencia, tasa_mutacion):
4     bases = ['A', 'C', 'G', 'T']
5     secuencia_mutada = list(secuencia)
6     for i in range(len(secuencia_mutada)):
7             opciones = [b for b in bases if
                  b != secuencia_mutada[i]]
8             secuencia_mutada[i] = rand.
                  choice(opciones)
9     return ''.join(secuencia_mutada)
```

De esta forma, nuestra implementación para mutar una secuencia de *ADN* de longitud $n \geq 1$ tiene que recorrerla por completo *(mediante el ciclo for)* y, en el peor de los casos si tenemos que mutar completamente la secuencia de *ADN (al tomar la tasa de mutación de $t = 1$)* vamos a tener que ejecutar las lineas 8 y 7.

Por lo tanto, la complejidad computacional de nuestra implementación en el peor de los casos es de $O(n)$.

Dado que con el índice de Jaccard estamos calculando la similitud entre dos conjuntos y nuestro objeto de estudio son genomas *(palabras finitas en el alfabeto Σ)*. Necesitamos una forma natural de convertir una secuencia de *ADN* a un conjunto para después hacer su respectiva comparación con el índice de Jaccard.

Para solucionar este problema vamos a seguir la idea presentada en el libro, de construir las **k-shingles** de las

secuencias de ADN.

Definición: Sea $g \in \Sigma^*$ una secuencia de ADN y $k \geq 1$ un entero. Entonces se definen los k-meros de la secuencia g como el conjunto de todas las sub-secuencias de longitud k de la secuencia g. Formalmente tenemos que:

$$\text{Seq}_k(g) := \{u \in \Sigma^* | u \text{ es sub-secuencia de } g \ , \ |u| = k\}$$

Con los siguientes teoremas vamos a analizar la complejidad espacial de un algoritmo que me permita construir el conjunto de k-meros dada una secuencia de ADN de longitud n.

Teorema: Sea $g \in \Sigma^*$ una secuencia de ADN de longitud $n \geq 1$ y $1 \leq k \leq n$, entonces el cardinal del conjunto de k-meros de la secuencia g:

$$|\text{Seq}_k(g)| \leq n - (k-1)$$

Demostración: Razonemos por inducción matemática sobre la longitud de la secuencia de ADN $g \in \Sigma^*$.

(Caso Base): Si la longitud de la secuencia g es 1, entonces el único valor que puede tomar k es 1. De esta forma, podemos concluir que:

$$\begin{aligned} |\text{Seq}_k(g)| &= 1 - (1-1) \\ &= 1 \end{aligned}$$

Luego para todas las secuencias de ADN de longitud 1 se cumple el enunciado.

(Paso Inductivo): Supongamos que para toda secuencia de ADN $f \in \Sigma^*$ de longitud $s < n$ se cumple que para todo $1 \leq k \leq s$:

$$|\text{Seq}_k(g)| \leq s - (k-1)$$

Veamos que para toda secuencia de ADN $g \in \Sigma^*$ de longitud n también se tiene el resultado.

(Caso 1): Si $k = n$ entonces el conjunto de k-meros de la secuencia $g \in \Sigma^*$ es:

$$\text{Seq}_k(g) = \{g\} \implies |\text{Seq}_k(g)| = 1$$

Y tenemos el siguiente desarrollo:

$$\begin{aligned} |\text{Seq}_k(g)| &= n - (k-1) \\ &= n - (n-1) \\ &= 1 \end{aligned}$$

Así pues, se cumple el enunciado para $k = n$.

(Caso 2): Si $1 \leq k < n$ entonces la secuencia de ADN $g \in \Sigma^*$ tiene la siguiente representación:

$$g = g_0 \ g_1 \ g_2 \ \cdots \ g_{n-2} \ g_{n-1}$$

Donde cada $g_i \in \Sigma = \{A, G, C, T\}$ para todo $1 \leq i \leq n-1$.

Si $u \in \Sigma^*$ es una sub-secuencia de g de longitud $1 \leq k < n$ entonces tenemos los siguientes casos:

Si la última base de u es g_{n-1} entonces:

$$u = g_{(n-1)-(k-1)} \cdots \ g_{n-2} \ g_{n-1}$$

Es la única sub-secuencia de g de longitud k que tiene como última base a g_{n-1}.

Si la última base de u no es g_{n-1} entonces consideramos la siguiente secuencia $g' \in \Sigma^*$ definida como:

$$g' = g_0 \ g_1 \ g_2 \ \cdots \ g_{n-2}$$

Claramente la longitud de g' es $|g'| = n-1$ y como $1 \leq k < n$ entonces u es una sub-secuencia de g' de longitud k. Por lo tanto:

$$u \in \text{Seq}_k(g')$$

Dado que los casos anteriores son excluyentes entonces tenemos el siguiente resultado:

$$|\text{Seq}_k(g)| \leq |\text{Seq}_k(g')| + 1$$

En virtud de la hipótesis de inducción podemos concluir:

$$|\text{Seq}_k(g')| \leq (n-1) - (k-1)$$

Así pues, para finalizar nuestro argumento inductivo tenemos el siguiente desarrollo:

$$\begin{aligned} |\text{Seq}_k(g)| &\leq |\text{Seq}_k(g')| + 1 \\ &\leq (n-1) - (k-1) + 1 \\ &\leq n - (k-1) \end{aligned}$$

Por lo tanto, el resultado se cumple para toda secuencia de ADN de longitud n y $1 \leq k \leq n$.

Nota: La igualdad $|\text{Seq}_k(g)| = n - (k-1)$ se tiene para todo $1 \leq k \leq n$ cuando todas las sub-secuencias de g de longitud k son diferentes.

Corolario: La complejidad espacial del siguiente algoritmo, encargado de construir el conjunto de los k-meros de una secuencia de ADN dada:

```
1  def k_meros(secuencia,k):
2    kmeros= set()
3    if k > len(secuencia):
4      return None
5    elif k == len(secuencia):
6      kmeros.add(secuencia)
7      return kmeros
8    else:
9      for i in range(len(secuencia)-(k-1)):
10       kmeros.add(secuencia[i:i+k])
11     return kmeros
```

es de $O(\ k(n-(k-1))\ )$ y, en su peor caso, si se toma $k \approx \frac{n}{2}$ tiene complejidad de $O(n^2)$

Demostración: Sea $g \in \Sigma^*$ una secuencia de ADN de longitud $n \geq 1$. Supongamos que el costo espacial *(en*

términos de memoria) de almacenar una carácter, en nuestro caso son los caracteres $\Sigma = \{A, G, C, T\}$ que representan las bases nitrogenadas, es de $O(1)$.

En virtud del teorema anterior, sabemos que la cantidad de k-meros diferentes de una secuencia de ADN de longitud n esta acodada por:

$$|\text{Seq}_k(g)| \leq n - (k-1)$$

Y dado que la longitud de cada k-mero es k, entonces la cantidad máxima de caracteres que guarda nuestro algoritmo es:

$$k \cdot |\text{Seq}_k(g)| \leq k \cdot (n - (k-1))$$

Así pues, la complejidad espacial de nuestro algoritmo es de $O(k \cdot (n - (k-1)))$.

Ahora considere el siguiente polinomio:

$$p(k) = k(n - (k-1))$$

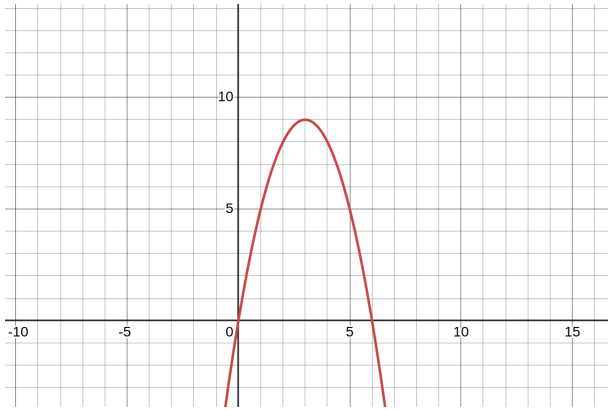


Figura 5. Polinomio $p(k) = k(n - k(-1))$ con $n = 5$

Este polinomio alcanza su valor máximo cuando $\frac{d}{dk}p(k) = 0$ entonces:

$$\frac{d}{dk}p(k) = n - 2k + 1$$

Por lo tanto, el argumento que maximiza la cantidad de memoria que necesita nuestro algoritmo es:

$$k_{\text{máx}} = \frac{n+1}{2}$$

Y la cantidad de memoria máxima que necesita nuestro algoritmo es:

$$\begin{aligned} p(k_{\text{máx}}) &= k_{\text{máx}}(n - (k_{\text{máx}} - 1)) \\ &= \frac{n+1}{2}\left(n - \left(\frac{n+1}{2} - 1\right)\right) \\ &= \frac{n+1}{2}\left(\frac{n+1}{2}\right) \\ &= \left(\frac{n+1}{2}\right)^2 \\ &= \frac{(n+1)^2}{4} \\ &= \frac{n^2 + 2n + 1}{4} \end{aligned}$$

De esta forma, en el peor de los casos nuestro algoritmo va a tener una complejidad espacial de $O(n^2)$.

Nota: la implementación del código se puede encontrar en el siguiente enlace Construcción del Conjunto de los k-meros.

Este último corolario, nos dice que para una secuencia de ADN suficientemente grande nuestro algoritmo de generación de los k-meros **no es viable desde el un punto de vista del costo de memoria**. Esta es una de las razones por las cuales se emplean firmas hash, en nuestro caso la firma **MinHash Sketch**. Que como veremos posteriormente, es una forma de simplificar la complejidad espacial del algoritmo de generación de k-meros, la cual tiene la propiedad fundamental de que **preserva el índice de Jaccard** al momento de comparar dos secuencias de ADN.

Para introducir las firmas MinHash Sketch, en primer lugar, necesitamos entender el concepto de la matriz característica de una familia de subconjuntos de un conjunto finito dado.

Definición: Sea $X = \{a_1, a_2, \ldots, a_n\}$ un conjunto y $S_1, S_2, \ldots, S_m \subseteq X$. Definimos la matriz característica del los subconjuntos $S_1, S_2, \ldots, S_m$ como la siguiente tabla:

Elemento	S_1	S_2	$\ldots$	S_m
a_1				
a_2				
			$\vdots$	
a_n				

Donde la tabla satisface las siguientes propiedades:

a) Cada columna representa un único subconjunto $S_1, S_2, \ldots, S_m$
b) Cada fila representa un único elemento del conjunto $X = \{a_1, a_2, \ldots, a_n\}$
c) La tabla se llena de la siguiente forma: Vamos a poner un 1 en la posición de la fila i-ésima y

columna j-ésima si y solo si el elemento $a_i \in S_j$. En otro caso, ponemos un 0.

Ejemplo: Si tomamos como conjunto universal al conjunto $X = \{a, b, c, d, e\}$ y consideramos la siguiente familia de subconjuntos de X:

$$\begin{aligned} S_1 &= \{a, d\} \\ S_2 &= \{c\} \\ S_3 &= \{b, d, e\} \\ S_4 &= \{a, c, d\} \end{aligned}$$

Entonces la matriz característica de la familia de subconjuntos S_1, S_2, S_3, S_4 es:

Elemento	S_1	S_2	S_3	S_4
a	1	0	0	1
b	0	0	1	0
c	0	1	0	1
d	1	0	1	1
e	0	0	1	0

Así pues, vemos que la matriz característica de la familia de subconjuntos S_1, S_2, S_3, S_4 nos permite reconocer rápidamente cuales elementos pertenecen a cada subconjunto solamente al leer su respectiva columna.

Ahora, para **introducir las funciones MinHash**, vamos a trabajar con el ejemplo anterior para dar una pequeña motivación antes de las definiciones formales.

Si tenemos el conjunto universal finito $X = \{a, b, c, d, e\}$ y consideramos una permutación $\pi_1 : X \to X$ definida como:

$$\begin{aligned} \pi_1(a) &= b \\ \pi_1(b) &= e \\ \pi_1(c) &= a \\ \pi_1(d) &= d \\ \pi_1(e) &= c \end{aligned}$$

Con esta permutación la matriz característica de los subconjuntos S_1, S_2, S_3, S_4 es:

Elemento	S_1	S_2	S_3	S_4
b	0	0	1	0
e	0	0	1	0
a	1	0	0	1
d	1	0	1	1
c	0	1	0	1

Así pues, definimos la función MinHash respecto a la permutación π como $h_{\pi_1} : \{S_1, S_2, S_3, S_4\} \to X$, la cual esta definida de la siguiente forma: El MinHash de cada subconjunto S_i, con $1 \leq i \leq 4$, es el primer elemento de la columna i-ésima que aparece con un 1 en la nueva matriz característica permutada por π_1. Con esta definición tenemos los siguientes valores para la función MinHash h_{π_1}:

$$\begin{aligned} h_{\pi_1}(S_1) &= a \\ h_{\pi_1}(S_2) &= c \\ h_{\pi_1}(S_3) &= b \\ h_{\pi_1}(S_4) &= a \end{aligned}$$

Como podemos observar esta función MinHash tiene una colisión para los subconjuntos S_1 y S_4. Si calculamos el índice de Jaccard de los subconjuntos tenemos que:

$$\begin{aligned} \text{Sim}(S_1, S_4) &= \frac{|S_1 \cap S_2|}{|S_1 \cup S_2|} \\ &= \frac{2}{3} \\ &\approx 0.66 \end{aligned}$$

Es decir, el índice de Jaccard nos dice que los conjuntos S_1 y S_2 tienen un grado de similitud de aproximadamente un 66 %.

¿Será que es una coincidencia de la permutación π_1 que tomamos?

Tomemos la siguiente permutación $\pi_2 : X \to X$ definida como:

$$\begin{aligned} \pi_2(a) &= e \\ \pi_3(b) &= c \\ \pi_4(c) &= a \\ \pi_5(d) &= d \\ \pi_6(e) &= b \end{aligned}$$

Entonces la matriz característica de los subconjuntos es:

Elemento	S_1	S_2	S_3	S_4
e	0	0	1	0
c	0	1	0	1
a	1	0	0	1
d	1	0	1	1
b	0	0	1	0

Y la función MinHash asociadad para la permutación π_2 toma los siguiente valores:

$$\begin{aligned} h_{\pi_2}(S_1) &= a \\ h_{\pi_2}(S_2) &= c \\ h_{\pi_2}(S_3) &= e \\ h_{\pi_2}(S_4) &= c \end{aligned}$$

En este caso, los conjuntos S_1 y S_4 no tienen colisión mediante la función MinHash. Sin embargo, la colisión se da para los conjuntos S_2 y S_4.

Si calculamos el índice de Jaccard de los conjuntos S_2 y S_4 tenemos:

$$\begin{aligned} \text{Sim}(S_2, S_4) &= \frac{|S_2 \cap S_4|}{|S_2 \cup S_4|} \\ &= \frac{1}{3} \\ &\approx 0.33 \end{aligned}$$

Es decir, los conjuntos S_2 y S_4 tienen una similitud de un 33 %, que es mucho menor que la similitud que tenían los conjuntos S_1 y S_4.

Una pregunta natural que nos surge con este ejercicio es:

¿Si tomamos aleatoriamente una permutación del conjunto X, cuántas de estas tienen una colisión con los subconjuntos S_1 y S_4?

Referencias

[1] B. D. Ondov, T. J. Treangen, P. Melsted, A. B. Mallonee, N. H. Bergman, S. Koren, y A. M. Phillippy, «Mash: fast genome and metagenome distance estimation using MinHash», Genome Biol., vol. 17, art. n.º 132, jun. 2016, doi: 10.1186/s13059-016-0997-x.

[2] J. Leskovec, A. Rajaraman, y J. D. Ullman, Mining of Massive Datasets. Palo Alto, CA, EE. UU., jul. 2019.

[3] A. Z. Broder, «On the resemblance and containment of documents», en Compression and Complexity of Sequences 1997 - Proceedings, IEEE, 1998, pp. 21–29.

[4] T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022.

[5] M. T. Goodrich, R. Tamassia y M. H. Goldwasser, *Data Structures and Algorithms in Python*. Hoboken, NJ, USA: Wiley, 2013.